

Тема 1.2.

Поняття алгоритму, властивості й способи задавання алгоритмів.

Розв'язування будь-якої задачі з застосуванням комп'ютера необхідно розбити на наступні етапи:

- розробка алгоритму розв'язування задачі;
- складання програми розв'язування задачі алгоритмічною мовою;
- ввід програми в комп'ютер;
- налагодження програми (виправлення помилок),
- виконання програми на комп'ютері;
- аналіз отриманих результатів.

Розглянемо перший етап розв'язування задачі – розробку алгоритму.

Поняття алгоритму

З давніх часів у математиці алгоритм уявляли як формальну інструкцію, яка визначає сукупність операцій і порядок їх виконання для розв'язування всіх задач деякого одного типу.

Для розв'язування ряду однотипних задач іноді доцільно використовувати суто механічні обчислювальні процеси. За їх допомоги шукані величини обчислюють послідовно з даних величин за певними правилами. Опис таких процесів прийнято називати алгоритмами. Взагалі, під алгоритмом інтуїтивно розуміють деякий формальний припис, діючи згідно з яким можна одержати розв'язок задачі.

Кожний зустрічається з алгоритмами зі школи. Правила, за якими виконують арифметичні дії з багатозначними числами, є найпростішими прикладами алгоритмів. Термін «алгоритм» походить від імені стародавнього узбецького математика Мухаммеда бен Муса аль Хорезмі, який ще в IX столітті сформулював такі правила.

У своєму розвитку математика нагромадила величезну кількість різних алгоритмів. Отримуючи відповідну інтерпретацію в конкретних

застосуваннях, вони становлять значну й найбільш істотну частину математичного апарата, використовуваного в техніці.

Алгоритм – чіткий опис послідовності дій, які необхідно виконати при розв’язуванні задачі [1].

Можна сказати, що алгоритм описує процес перетворення вхідних даних у результати, оскільки для отримання розв’язку будь-якої задачі необхідно:

увести вхідні дані;

перетворити вхідні дані в результати (вихідні дані);

вивести результати.

Розробка алгоритму розв’язування задачі – це розбиття задачі на послідовно виконувані етапи, причому результати виконання попередніх етапів можуть використовуватися при виконанні наступних. При цьому повинні бути чітко зазначені як зміст кожного етапу, так і порядок виконання етапів. Окремий етап алгоритму є або іншою, більш простою задачею, алгоритм розв’язування якої відомий (розроблений заздалегідь), або повинен бути досить простим і зрозумілим без пояснень.

У наш час виникла потреба у глибокому вивченні й осмисленні алгоритмів, головним чином у зв’язку з проблемою алгоритмічної нерозв’язності. Річ у тім, що при спробах розв’язати ряд задач виникли труднощі, які не вдалося подолати, незважаючи на довготривалі й завзяті зусилля багатьох великих математиків.

Згодом з’ясувалось, що існує коло задач, які алгоритмічно важко розв’язні (так звані *NP*-повні задачі).

Для вирішення проблем, що виникають при розробці алгоритмів, зокрема, для ефективного визначення *NP*-повних задач, у 30-ті роки була створена нова наука – «Теорія алгоритмів».

Чисельні та логічні алгоритми

Алгоритми можуть бути чисельними й логічними.

Алгоритми, які представляють розв'язування задачі у вигляді арифметичних дій над числами, називають *чисельними алгоритмами*.

Типовим прикладом чисельного алгоритму є алгоритм Евкліда для знаходження найбільшого спільного дільника двох заданих додатних цілих чисел a і b [2].

Довідково. Найбільшим спільним дільником (НСД) чисел a і b називають деяке найбільше число c , на яке обидва числа a і b діляться без остачі.

Опис алгоритму Евкліда

Алгоритм Евкліда складається з наступної системи послідовних вказівок:

1. Вводимо числа a і b .
2. Перевіряємо умову $a = b$. Якщо ця умова вірна, то розв'язок знайдений. Обидва числа є НСД одне для одного. Переходимо до пункту 6. Якщо $a \neq b$, то переходимо до 3.
3. Якщо $a < b$, то значення a та b міняємо місцями: $a \leftrightarrow b$.
4. Якщо a ділиться без остачі на b , то число b і є НСД. Переходимо до пункту 6, інакше переходимо до пункту 5.
5. Число a заміняємо на остачу від ділення і переходимо до пункту 3.
6. Вивід результатів.

Приклад 1.1. Застосування алгоритму Евкліда

Завдання. Знайти НСД для 30 і 18.

Розв'язок. $30/18 = 1$ (остача 12).

$$18/12 = 1 \text{ (остача 6).}$$

$$12/6 = 2 \text{ (остача 0).}$$

Кінець: НСД – це дільник. НСД (30, 18) = 6.

Отже, після п'ятої вказівки потрібно щоразу повертатися до третьої, допоки не буде виконана четверта вказівка. Хоча заздалегідь не відомо, скільки буде потрібно таких циклічних переходів, але ясно, що для будь-яких двох чисел мета буде досягнута за скінченну кількість кроків.

Логічні алгоритми, як правило, полягають у маніпулюванні даними з метою встановлення або визначення деякої закономірності. До логічних

алгоритмів можна віднести алгоритми класифікації, сортування, планування та ін. [3].

Приклад 1.2. Логічний алгоритм. Найпростішим прикладом логічного алгоритму є алгоритм сортування. Розглянемо, наприклад, сортування Хоара.

I етап алгоритму Хоара

Номер кроку	$i \rightarrow$						$\leftarrow j$	Примітки
	0	1	2	3	4	5		
	40	11	83	21	75	64		Початковий список
1	40					64		$k_0 < k_j$ ОК
2	40				75			$k_0 < k_j$ ОК
3	40			21				$k_0 > k_j$ Обмін $k_0 > k_i$ ОК
	21			40				
3	21	11	83	40	75	64		Примітки
4		11		40				$k_0 > k_i$ ОК
5			83	40				$k_0 < k_i$ Обмін $k_0 > k_i$ ОК
			40	83				
6	21	11	40	83	75	64		Кінець етапу

II етап алгоритму Хоара

Номер кроку	$i_1 \rightarrow$		$\leftarrow j_1$	Примітки	$i_2 \rightarrow$				$\leftarrow j_2$	Примітки
	0	1			0	1	2	3		
	21	11		Поч. спис.	40	83	75	64		Поч. спис.
7	21	11		$k_0 > k_j$ Обмін	40	83		64		$k_0 > k_j$ Обмін
	11	21		$k_0 > k_i$ ОК		64		83		$k_0 > k_i$ ОК
8					40		75	83		$k_0 > k_i$ ОК
9	11	21		Кінець алг.	40	64	75	83		Кінець алг.

Рис. 1.1. Алгоритм сортування Хоара

Прийmemo перший елемент послідовності за базовий ключ, виділимо його червоним кольором і позначимо $k_0 = 40$. Установимо два покажчики i та j , з яких i починає відлік ліворуч ($i = 1$), а j – праворуч ($j = n$). Порівнюємо базовий ключ k_0 і поточний ключ k_j . Якщо $k_0 < k_j$, то встановлюємо $j := j - 1$ й проводимо наступне порівняння k_0 й k_j . Продовжуємо зменшувати j доти, поки не досягнемо умови $k_0 > k_j$. Після цього міняємо місцями ключі k_0 й k_j (див. крок 3 на рис. 1.1).

Тепер починаємо змінювати індекс $i := i + 1$ і порівнювати елементи k_i й k_0 . Продовжуємо збільшення i доти, поки не досягнемо умови $k_i > k_0$, після чого відбувається обмін k_i і k_0 (див. крок 5). Знову вертаємося до індексу j , зменшуємо його. Почергово зменшуючи j та збільшуючи i , продовжуємо цей процес із обох кінців до середини доти, поки не одержимо $i = j$ (див. крок 6).

Уже на першому етапі наявні два факти:

по-перше, базовий ключ $k_0 = 40$ зайняв своє постійне місце у послідовності для сортування;

по-друге, усі елементи ліворуч від k_0 будуть меншими за нього, а праворуч – більшими за нього.

Таким чином, по закінченні першого етапу маємо:

<u>21, 11, 40, 83, 75, 64</u>	
Права частина	Ліва частина

Етап 7 починають з вибору базових ключів окремо для лівої й правої частини. Потім виконують незалежно два сортування: сортування лівої частини й сортування правої частини.

Послідовні та паралельні алгоритми [4]

Послідовний алгоритм – це алгоритм, у якому кожний наступний етап використовує результати деякої підмножини попередніх етапів. Традиційний підхід одержання паралельного алгоритму з послідовного полягає в пошуку незалежних ділянок з метою організації їх паралельного виконання.

Приклад 1.3. Послідовний алгоритм

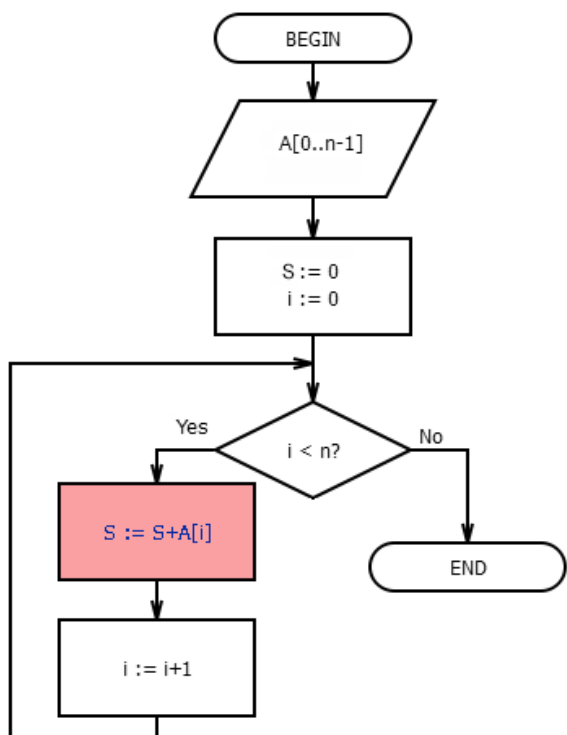


Рис. 1.2. Послідовний алгоритм додавання n чисел

Нехай даний звичайний алгоритм підсумовування (додавання) n чисел, коли на кожному кроці до часткової суми додають черговий доданок (рис. 1.2). Такий алгоритм є виключно послідовним. Критичною операцією даного алгоритму є додавання чергового числа до проміжної суми: $S := S + A[i]$.

Додавання даного числа може виконуватися тільки після завершення операції підсумовування з попереднім.

Даний простий приклад показує,

що не завжди для одержання паралельного алгоритму можна використовувати методику розпаралелювання, що полягає в пошуку незалежних ділянок. Якщо розпаралелювання виявилось нерезультативним, то це ще не означає, що принципово не можна одержати результат, використовуючи паралельні обчислення.

Паралельний алгоритм – алгоритм, який може бути реалізований по частинах на різних обчислювальних засобах з наступним об'єднанням часткових результатів для одержання глобального розв'язку задачі [5].

Приклад 1.4. Паралельний алгоритм

Розглянемо приклад побудови паралельного варіанта алгоритму підсумовування n чисел (рис. 1.3). Розіб'ємо всі доданки на пари й здійснимо підсумовування двох чисел усередині кожної пари. Усі ці операції незалежні. Отримані часткові суми також розіб'ємо на пари й знову здійснимо підсумовування двох чисел усередині кожної пари. Знову всі операції незалежні. Уся сума буде отримана через $\log_2 n$ кроків.

У даному алгоритмі вже є значний ресурс паралелізму, хоча він не рівномірний. На першому часовому кроці можуть бути використані $n/2$ процесорів, на другому $n/4$ і т. д. Названий цей алгоритм процесом здвоювання.

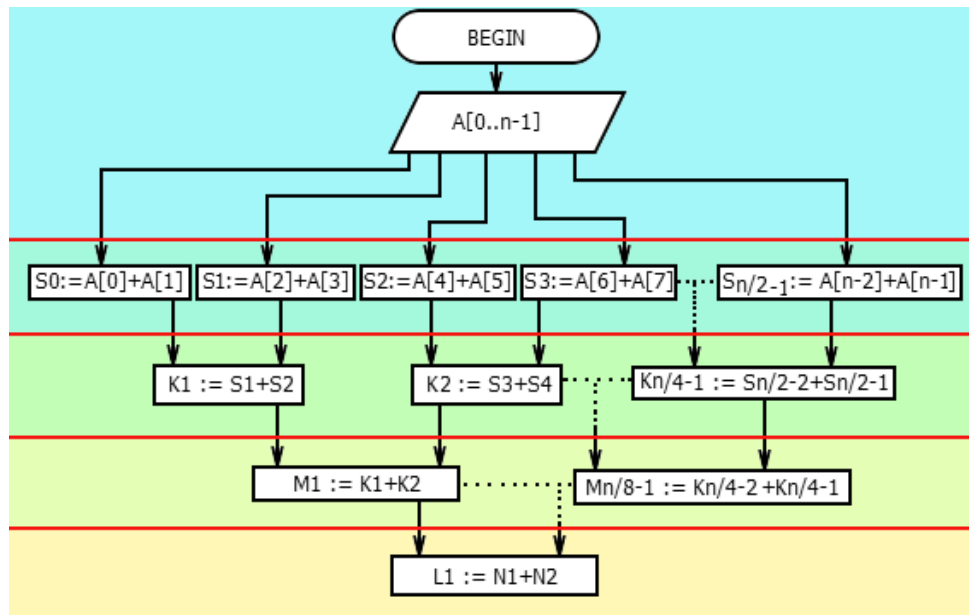


Рис. 1.3. Паралельний алгоритм додавання n чисел

Зазначимо, що обидва алгоритми ґрунтуються на реалізації математично еквівалентних виразів підсумовування чисел, але вони мають різні властивості, принаймні, з точки зору паралельних обчислень. Насправді, у них багато й інших відмінностей: вони по-різному реагують на помилки округлення, по-різному використовують пам'ять і т. п. Тому ці алгоритми слід вважати принципово різними, незважаючи на те, що вони математично еквівалентні!

Саме цей факт став причиною активізації досліджень в області розробки нових обчислювальних методів, орієнтованих на реалізацію в паралельних обчислювальних системах, і розвитку теорії паралельних алгоритмів, які базуються на паралельних методах.

Загальні властивості алгоритмів

Досвід розробки й застосування алгоритмів підказує ряд загальних властивостей будь-яких алгоритмів:

1. **Дискретність алгоритму:** будь-який алгоритм можна розглядати як процес послідовної побудови величин, що йде в дискретному часі, виконуючи набір певних інструкцій, який називають програмою.

2. **Масовість алгоритму:** алгоритм придатний для розв'язування не конкретної визначеної однієї задачі, а цілого класу однотипних задач. У цьому аспекті таблиця множення не є алгоритмом, а множення стовпчиком багатозначних чисел – алгоритм. Наприклад, в алгоритмі Евкліда числа a і b вибирають з нескінченної зліченної множини цілих чисел.

3. **Детермінованість алгоритму:** після виконання чергового етапу однозначно визначено, що робити на наступному етапі.

4. **Елементарність кроків алгоритму:** розв'язування задачі розбивають на етапи, кожний з яких повинен бути простим і локальним. Це означає, що відповідна операція повинна бути елементарною для виконавця алгоритму (людини або машини). Наприклад, операції, наявні в алгоритмі Евкліда, – порівняння, ділення та перестановка чисел – є досить стандартними й звичними. У той же час сам алгоритм Евкліда може бути елементарною операцією більш складного алгоритму.

5. **Результативність алгоритму:** алгоритм через скінченну кількість кроків повинен привести до зупинки, яка свідчить про досягнення необхідного результату. Так, при пошуку шляху в лабіринті зупинка настає або на досягнутому майданчику, або при поверненні до місця старту, якщо зазначена мета не досягнута. Зокрема, кожен, хто пред'являє алгоритм розв'язування деякої задачі, зобов'язаний показати, що алгоритм зупиняється після скінченної кількості кроків. Ще говорять, що алгоритм збігається для будь-якого x з області визначення функції $f(x)$. Однак перевірити результативність (збіжність алгоритму) набагато важче, ніж інші його властивості.

Способи задавання алгоритмів

Існує досить велика кількість формальних засобів, що дозволяють однозначно задавати сучасні алгоритми. Однак у рамках даного курсу будемо розглядати тільки традиційні, найпоширеніші способи:

1. *Спосіб задавання алгоритму у вербальній (словесній) формі*

Це найдавніший спосіб задавання алгоритмів. Перші алгоритми формулювали винятково словесно. Прикладом словесної форми задавання може служити алгоритм Евкліда, розглянутий вище. Як правило, словесний опис складається з пунктів, у кожному з яких чітко формулюють найпростішу дію. Потім дають вказівку про подальші дії.

2. *Спосіб задавання алгоритму в графічній формі*

Блок-схема – це графічне зображення алгоритму. При її побудові зміст кожного кроку алгоритму записують у довільній формі усередині блоку, представленого геометричною фігурою. Порядок виконання кроків вказують за допомогою стрілок, що з'єднують блоки. Використання різних геометричних фігур відображає різний характер виконуваних дій.

3. *Спосіб задавання алгоритму у вигляді псевдокоду*

Псевдокод – це опис алгоритмів умовною алгоритмічною мовою, що включає елементи мови програмування, фрази природної мови, загальноприйняті математичні позначення й ін. Такий спосіб опису часто використовується в навчальній і науковій літературі з метою забезпечення компактного представлення алгоритму.

4. *Спосіб задавання алгоритму у вигляді комп'ютерної програми*

Для задавання алгоритму у вигляді програми використовують конструкції конкретної мови програмування. Представлений у вигляді програми алгоритм після компіляції перетворюється на файл, що виконується комп'ютером.

Зображення алгоритму у вигляді блок-схеми

Блок-схемою називають наочне графічне зображення алгоритму, у якому окремі етапи алгоритму зображують за допомогою різних геометричних фігур – блоків, а зв'язки між етапами (послідовність виконання етапів) вказують за допомогою стрілок, які з'єднують ці фігури. Конфігурація й розміри блоків, а також порядок графічного оформлення блок-схем регламентовані ДЕРЖСТАНДАРТ 10.002-80 і ДЕРЖСТАНДАРТ 10.003-80 «Схеми алгоритмів і програм». Блоки супроводжують написами.

У таблиці 1.1 наведені найбільш часто використовувані блоки, зображені елементи зв'язків між ними й дано коротке пояснення до них. Блоки й елементи зв'язків називають елементами блок-схем.

Представлених у таблиці елементів цілком достатньо для зображення алгоритмів, які необхідні при виконанні студентських робіт.

При з'єднанні блоків слід використовувати тільки вертикальні й горизонтальні лінії потоків.

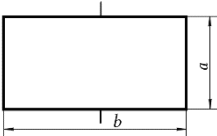
Горизонтальні потоки, що мають напрямок праворуч ліворуч (справа наліво), і вертикальні потоки, що мають напрямок знизу вгору, повинні бути обов'язково позначені стрілками.

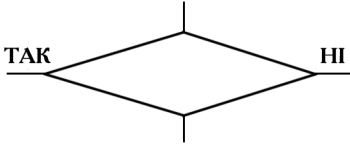
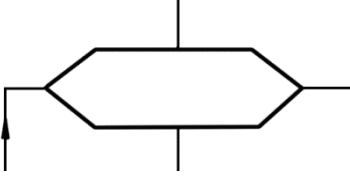
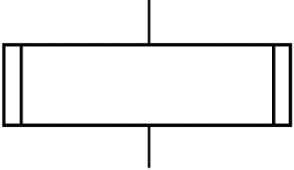
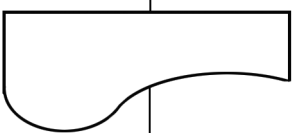
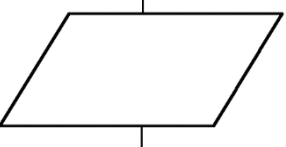
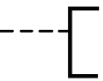
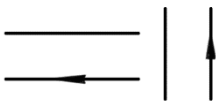
Інші потоки можуть бути позначені або залишені непозначеними.

Лінії потоків мають бути паралельні лініям зовнішньої рамки або границям аркуша.

Таблиця 1.1

Основні елементи блок-схем

Назва	Елемент	Коментар
Процес		Обчислювальна дія або послідовність обчислювальних дій

Назва	Елемент	Коментар
Розв'язування		Перевірка умови
Модифікація		Заголовок циклу
Визначений процес		Звертання до процедури
Документ		Вивід даних, друк даних
Введення/Виведення		Ввід/Вивід даних
З'єднувач		Розрив лінії потоку
Початок, Кінець		Початок, кінець, пуск, зупинка, вхід і вихід у допоміжних алгоритмах
Коментар		Використовують для розміщення написів
Горизонтальні й вертикальні потоки		Лінії зв'язків між блоками, напрямки потоків
Злиття		Злиття ліній потоків
Міжсторінковий з'єднувач		Перехід на інший лист

Відстань між паралельними лініями потоків повинна бути не менше 3 мм, між іншими елементами схеми – не менше 5 мм.

Горизонтальний і вертикальний розміри блоку мають бути кратні 5 (ділитися на 5 націло). Співвідношення горизонтального й вертикального розмірів блоку $b/a = 1.5$ є основним. При ручному виконанні креслення блоку припустиме співвідношення $b/a = 2$.

Блоки «Початок», «Кінець» і «З'єднувач» мають висоту $a/2$, тобто вдвічі меншу за основну висоту блоків.

Для розміщення блоків рекомендується поле аркуша розбивати на горизонтальні й вертикальні (для схем, що розгалужуються) зони.

Для зручності опису блок-схеми кожний її блок слід пронумерувати, використовувати наскрізну нумерацію блоків. Номер блоку розташовують у розриві в лівій верхній частині рамки блоку.

Види алгоритмів

По характеру зв'язків між блоками розрізняють алгоритми лінійної, циклічної структури, та структури, що розгалужується.

Лінійний алгоритм

Лінійний алгоритм – це алгоритм, у якому блоки виконуються послідовно зверху вниз від початку до кінця.

На рис. 1.4 наведений приклад блок-схеми алгоритму обчислення периметра P і площі S квадрата зі стороною довжини A .

Блок-схема алгоритму складається із шести блоків. Виконання алгоритму починається із блоку 1 «Старт». Цей блок символізує включення комп'ютера, налаштування його на виконання алгоритму й виділення пам'яті під усі змінні, які задіяні в алгоритмі. В алгоритмі рис. 1.4 таких змінних три: A , P , S . Отже, під кожен зі змінних алгоритмом буде виділено по одній комірці пам'яті. На цьому блок 1 буде відпрацьований.

Як видно з рис. 1.4, блок 1 зв'язаний вертикальною лінією потоку з блоком 2. Ця лінія не має стрілки, що вказує напрямок потоку. Отже, цей

потік спрямований униз. Таким чином, після виконання блоку 1 керування буде передано на блок 2. Блок 2 «Введення даних» (див. табл. 1.1) показує, що змінній A слід присвоїти значення. Це означає, що в комірку, відведену комп'ютером під цю змінну, потрібно помістити константу. На реальному комп'ютері ця константа може бути введена у різний спосіб. Спосіб залежить від того, як запрограмований даний фрагмент. Можна, наприклад, передбачити введення константи з клавіатури або одержати його із задалегідь підготовленого файлу. Можливо, ця константа буде отримана через зовнішні джерела даних, наприклад, від фізичної установки, підключеної до комп'ютера.

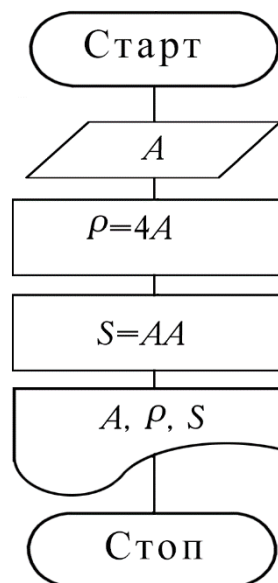


Рис. 1.4. Приклад лінійного алгоритму

Для даного прикладу спосіб передачі константи не має значення, важливо лише те, що при виконанні блоку 2 у комірку з адресою A буде занесена конкретна константа. Нехай такою константою є число 5.

Далі керування по лінії потоку передається до блоку 3 «Процес». У цьому блоці при виконанні розміщеної в ній команди число 4 множиться на константу, поміщену в комірку A (тобто 5), і результат (тобто 20)

присвоюється змінній P (тобто константа 20 записується в комірку за адресою P). Після виконання цих операцій керування передається до блоку 4.

У блоці 4 аналогічним чином проводиться множення значень змінної A і результат (константа 25) присвоюється змінній S (у комірку за адресою S буде занесена константа 25). Після цього виконується перехід до блоку 5.

При виконанні команд блоку 5 виводяться (наприклад, на екран, папір, у зовнішній файл і т. д.) значення змінних A , P , S , які збереглися у відповідних комірках до цього моменту. Зрозуміло, що для конкретного прикладу $A = 5$ будуть виведені константи 5, 20, 25, тобто довжина сторони квадрата, його периметр і площа. Далі керування передається останньому блоку 6.

У блоці 6 «Стоп» проводиться звільнення комірок пам'яті, які були зарезервовані під змінні A , P , S , і алгоритм закінчує роботу.

Алгоритми, що розгалужуються

На практиці алгоритми лінійної структури трапляються вкрай рідко. Частіше необхідно організувати процес, який залежно від деяких умов проходить по тій або іншій гілці алгоритму. Такий алгоритм називають алгоритмом, що розгалужується.

У блок-схемах розгалуження починається на виходах елемента «Розв'язок», за допомогою якого в алгоритмі виконується перевірка будь-якої умови. Кількість гілок тим більша, чим більше умов, що перевіряються.

Для пояснення розглянемо розв'язування задачі знаходження значення функції $z = x/y$.

На перший погляд здається, що алгоритм розв'язування цієї задачі має лінійну структуру. Однак, якщо врахувати, що ділити на нуль не можна через переповнення комірки, то алгоритм потребує ускладнення.

По-перше, потрібно з обчислень виключити варіант $x = 0$.

По-друге, потрібно проінформувати користувача алгоритму про помилку, яка виникла.

Якщо цього не зробити, то при обчисленнях може виникнути аварійний вихід до одержання результату. У професійній практиці аварійні завершення

вкрай небажані, оскільки до цього моменту вже може бути накопичена певна кількість результатів, які виявляться неопрацьованими й просто пропадуть. Можна привести інші приклади, коли аварійна зупинка комп'ютера може спричинити набагато серйозніші наслідки.

Розв'язок задачі представлений блок-схемою на рис. 1.5.

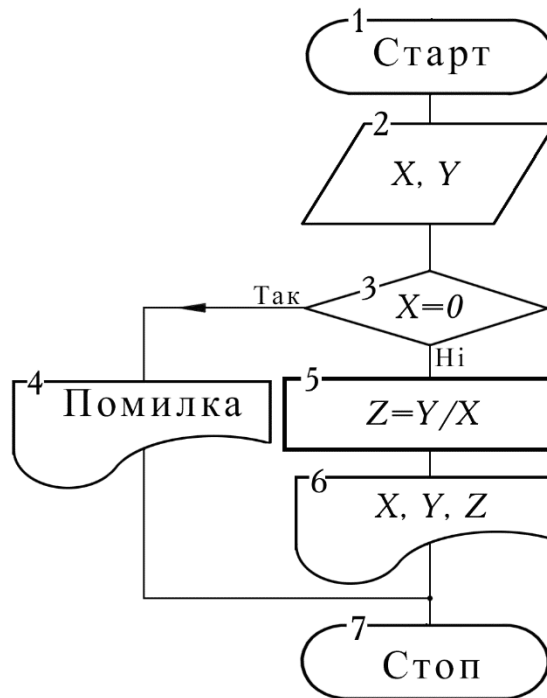


Рис. 1.5. Алгоритм, що розгалужується

Блок-схема складається з 7 блоків. Після початку роботи алгоритм у блоці 2 вимагає введення аргументів X і Y . Потім у блоці 3 проводиться перевірка умови $X = 0$. Тут алгоритм перевіряє, чи дорівнює нулю константа, уведена в комірку з адресою X . Результатом такої перевірки є відповідь «Так» або «Ні». Залежно від цієї відповіді виконання алгоритму піде по одній або іншій гілках. Якщо результат перевірки виявиться негативним, то на X можна ділити, і керування передається блоку 4.

У блоці 4 буде отриманий результат Z , потім у блоці 6 значення всіх трьох змінних будуть надруковані й у блоці 7 алгоритм закінчить роботу. Якщо ж відповідь виявиться позитивною, то керування буде передано блоку 4.

Виконуючи команду блоку 4, алгоритм виведе повідомлення «Помилка» і потім закінчить роботу в тому ж блоці 7.

Циклічний алгоритм

Часто при розв'язуванні задач доводиться повторювати виконання операцій по тих самих залежностях при різних значеннях змінних, що входять у ці залежності, і робити багаторазовий прохід по тих самих ділянках алгоритму. Такі ділянки називають циклами.

Цикл – різновид керуючої конструкції, призначеної для організації багаторазового виконання одних і тих самих ділянок алгоритму при різних значеннях змінних.

Алгоритми, що містять цикли, називають *циклічними*.

Використання циклів суттєво скорочує обсяг алгоритму. Етапи організації циклу:

- підготовка (ініціалізація) циклу;
- виконання обчислень циклу (тіло циклу);
- модифікація параметрів;
- перевірка умови закінчення циклу.

Розрізняють цикли з наперед відомою і наперед невідомою кількістю проходів.

Цикл називають *детермінованим*, якщо число повторень тіла циклу заздалегідь відомо або визначено.

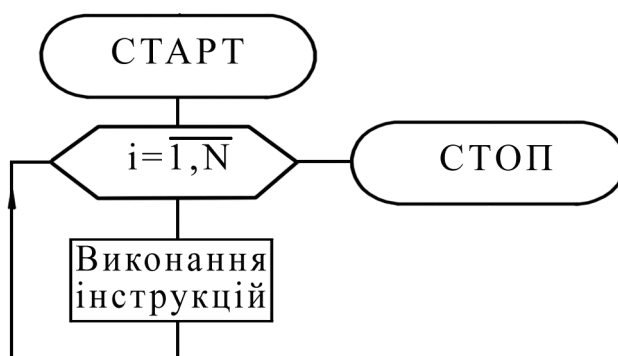


Рис. 1.6. Алгоритм з детермінованим циклом

Цикл називають ітераційним, якщо число повторень тіла циклу заздалегідь невідомо, а залежить від значень параметрів (деяких змінних), що беруть участь в обчисленнях.

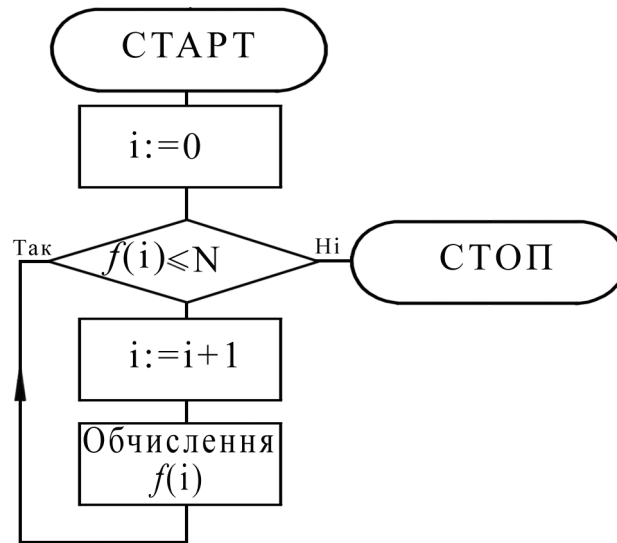


Рис. 1.7. Алгоритм із ітераційним циклом

Приклад 1.5. Розглянемо приклад алгоритму з циклом, що має наперед невідому кількість проходів. Для цього розв’яжемо наступну задачу.

Указати найменшу кількість членів ряду натуральних чисел 1, 2, 3, ..., сума яких більша за число K .

Блок-схема алгоритму розв’язування цієї задачі наведена на рис. 1.8. Вона складається з восьми блоків.

Після початку роботи в блоці 2 уводиться значення числа K . Далі в блоці 3 змінна i одержує значення 1, тобто значення, з якого почнеться відлік натуральних чисел. Змінна S , призначена для накопичення суми цих чисел, перед початком підсумовування одержує значення 0. Після цього керування передається блоку 5.

У ньому при виконанні команди $S = S + i$ проводиться додавання вмісту комірок S та i , а результат записується в комірку S . Оскільки до операції додавання було $S = 0$, $i = 1$, то після виконання операції $S = 1$. При записі нового значення старий вміст комірки S (нуль) стирається, а на його місце записується число 1.

Зауважимо, що якби до цієї операції в блоці 3 не була виконана команда $S = 0$ (записати нуль у комірку S), то при обчисленні суми $S+1$ виникла б помилка, оскільки із комірки S була б взята константа, яка з'явилася там після розподілу пам'яті.

Після підсумовування першого члена послідовності в блоці 6 виконується перевірка умови про перевищення сумою S заданого числа K .

Якщо умова 6 не виконається, то проводиться перехід до блоку 4, де при виконанні операції значення змінної збільшується на 1 і дорівнює 2. Тепер алгоритм знову повернеться до блоку 5 і до старого значення суми додасть новий член 2. Після цього сума дорівнюватиме 3.

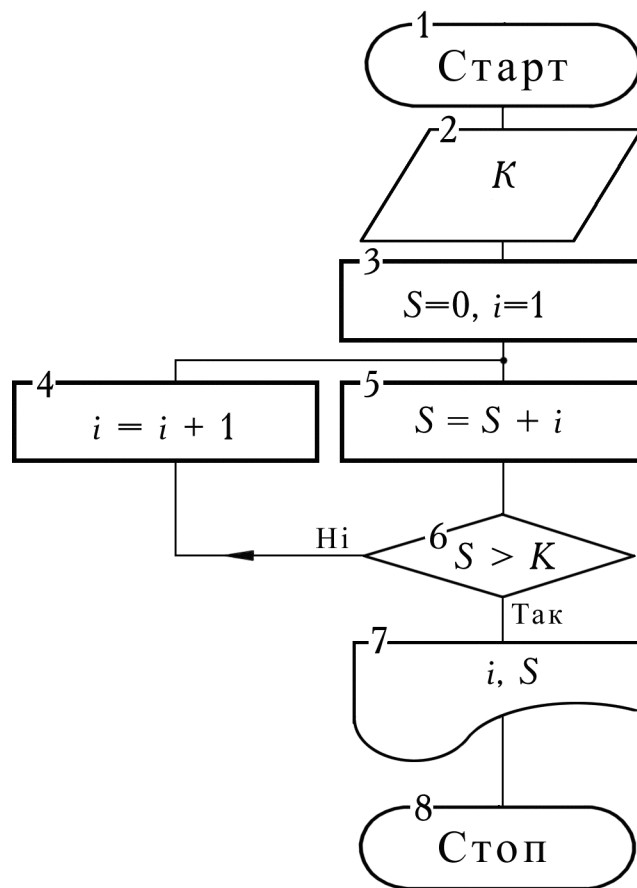


Рис. 1.8. Приклад циклічного алгоритму

У блоці 6 знову перевіряється умова одержання необхідної суми і т. д. Ланцюжок блоків 4–5 буде оброблятися знову й знову до того моменту, коли одного разу при певному значенні змінної i , нарешті, виконається умова $S > K$, тобто сума, що накопичується в такому циклі, уперше перевищить задане

значення K . Змінна i , значення якої при черговому проході ланцюжка цих блоків збільшується на 1, відіграє роль лічильника цього циклу.

Далі проводиться перехід до блоку 7, де надрукується значення кількості членів ряду (взяте й надрукуване число із комірки i , яке там зберігається в момент виконання умови), суми S та в блоці 8 алгоритм закінчить роботу.

1.1.6. Приклади задавання алгоритму Евкліда

Блок-схема алгоритму Евкліда

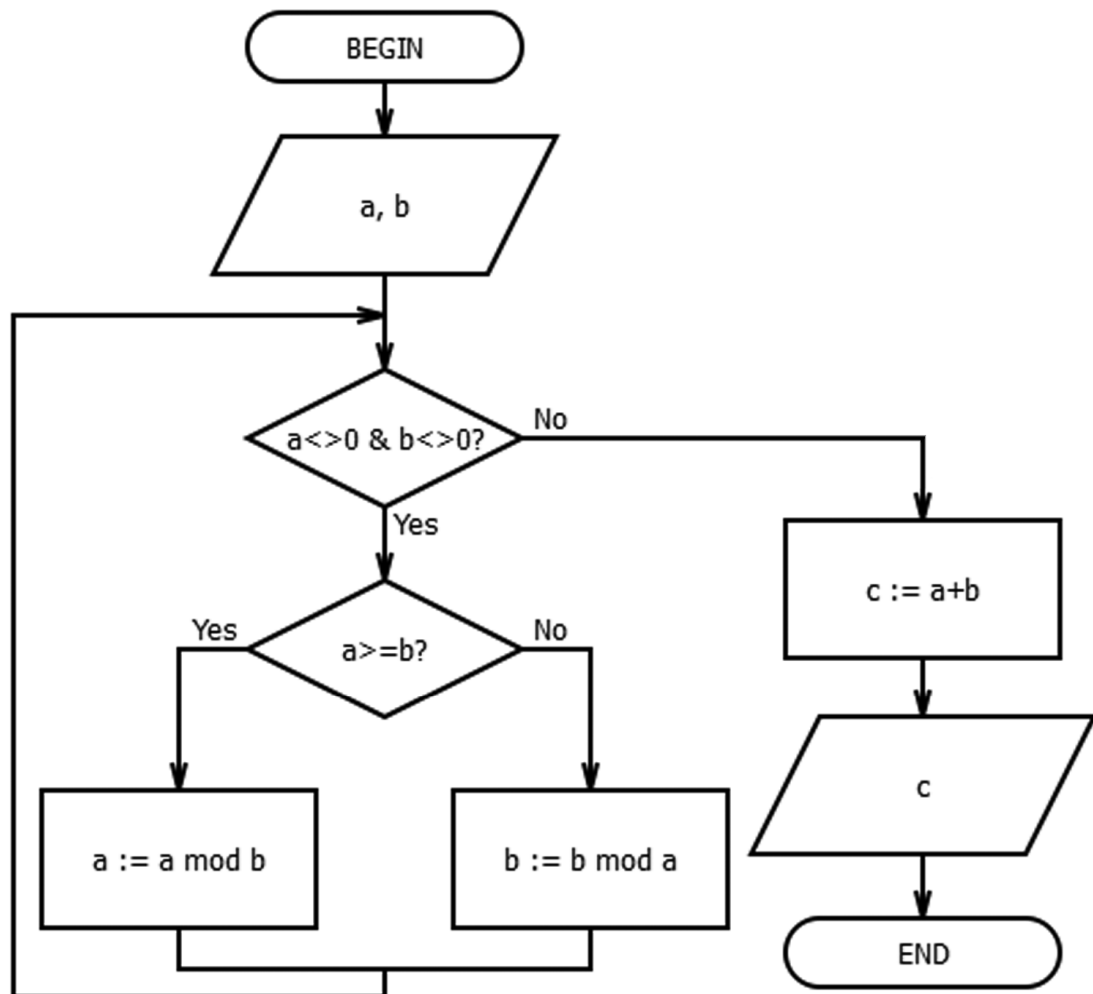


Рис. 1.9. Блок-схема алгоритму Евкліда

Псевдокод алгоритму Евкліда

1. Input a, b .
2. If $a < b$ then $a \leftrightarrow b$ else знайти q, r з $a = bq + r$.
3. If $r = 0$ then $c := b$ else $a := b$; $b := r$; goto 2.

Приклад 1.6. Програми за алгоритмом Евкліда

C++ <pre>while (b) { a %= b; swap (a,b); } gcd = a;</pre>	Python <pre>def gcd(a, b): return b and gcd(b, a%b) or a</pre>
Java <pre>public class GCD { public static int gcd(int a,int b) { while (b !=0) { int tmp = a%b; a = b; b = tmp; } return a; } }</pre>	

Інші способи задавання алгоритмів

Мережі Петрі

Мережу Петрі задають у вигляді кортежу [6]:

$$\Phi = (P, T, F, M),$$

де $P = \{p_1, p_2, \dots, p_n\}$ – скінченна множина позицій, $n > 0$;

$T = \{t_1, t_2, \dots, t_m\}$ – скінченна множина переходів, $m > 0$;

$F \subseteq \{P \times T\} \cup \{T \times P\}$ – скінченна множина ребер, які з'єднують переходи й позиції;

M – вектор маркувань.

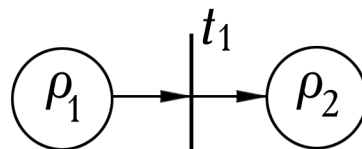


Рис. 1.10. Найпростіша мережа Петрі

На рис. 1.11 наведена мережа Петрі з такими параметрами:

$F = I \cup O$, $I \subset \{P \times T\}$ – підмножина вхідних ребер;

$O \subset \{T \times P\}$ – підмножина вихідних ребер;

$$P = \{p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9\}$$

$$T = \{t_1, t_2, t_3, t_4, t_5, t_6\}$$

$$I = \{(p_1, t_1), (p_2, t_2), (p_3, t_3), (p_6, t_4), (p_4, t_4), (p_8, t_5), (p_5, t_5), (p_7, t_6)\}$$

$$O = \{(t_1, p_2), (t_1, p_3), (t_1, p_4), (t_2, p_5), (t_3, p_6), (t_4, p_7), (t_5, p_8), (t_6, p_9)\}$$

$$F = \begin{pmatrix} & p_1 & p_2 & p_3 & p_4 & p_5 & p_6 & p_7 & p_8 & p_9 \\ t_1 & 1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 \\ t_2 & 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ t_3 & 0 & 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 \\ t_4 & 0 & 0 & 0 & 1 & 0 & 1 & -1 & 1 & 0 \\ t_5 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & 0 \\ t_6 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & -1 \end{pmatrix}.$$

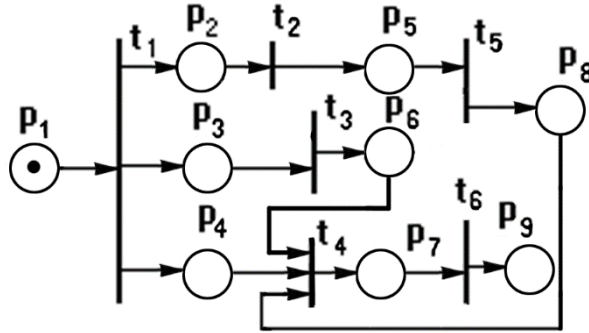


Рис. 1.11. Приклад мережі Петрі

Динаміка мережі Петрі

Динаміка мереж Петрі базується на двох основних правилах.

Правило активації

Перехід $t \in T$ активується відповідним маркуванням μ його вхідних позицій тоді й тільки тоді, коли вони містять задану кількість міток.

Активация переходів створює передумови їх спрацювання, але не кожний активований перехід може спрацювати.

Правило спрацювання

Спрацювання переходу t , активованого маркуванням $\mu^{(\tau)}$, породжує нове маркування мережі $\mu^{(\tau+1)}$ шляхом вилучення міток із вхідних позицій і розміщення деякої кількості міток на вихідних позиціях.

Контрольні запитання

1. Опишіть загальні властивості послідовних алгоритмів.
2. Нехай необхідно побудувати такі алгоритми:
 - Обчислення суми перших n елементів геометричної прогресії.
 - Сортуння масиву вставками.
 - Пошук максимального числа в тривимірному масиві.
 - Знаходження коренів нелінійного рівняння 5-го степеня.Визначте, які з цих алгоритмів є чисельними, а які – логічними.
3. Чи можна побудувати паралельний алгоритм додавання N чисел?
4. Нехай потрібно обчислити вираз:

$$f(x, y) = \begin{cases} y/x, & x > y, \\ y/x, & x \leq y \end{cases} - 10 \leq x \leq 10, -5 \leq y \leq 5, \text{ де } x, y - \text{цілі числа.}$$

Запишіть псевдокод алгоритму обчислення виразу та задайте алгоритм графічно.

5. Алгоритм Евкліда є чисельним чи логічним алгоритмом? Запишіть послідовність дій алгоритму Евкліда при $a = 8$ і $b = 12$.